

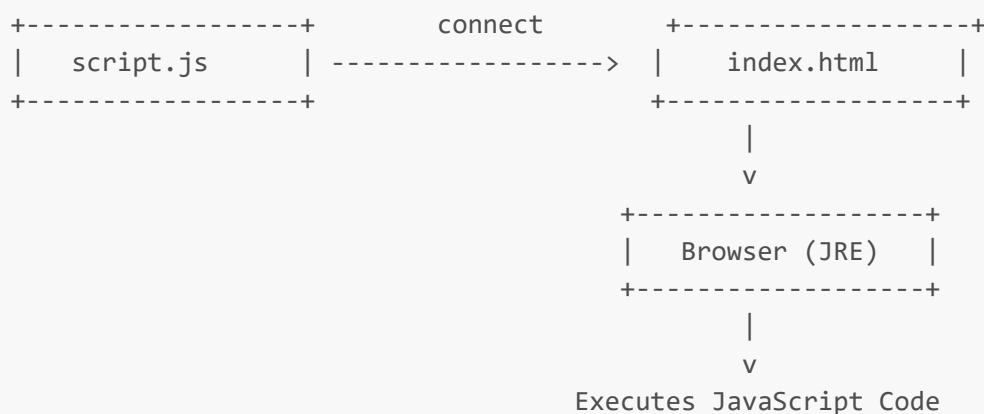
JavaScript: How It Actually Works

This note is a comprehensive breakdown of JavaScript's working environment, engine, and Node.js evolution.

JavaScript Working Mechanism

1. JavaScript Runtime Environment (JRE)

When `script.js` is connected to `index.html` and opened in a browser, it runs using the **JavaScript Runtime Environment** provided by the browser.



2. Web APIs (provided by Browser)

setInterval	PROMISE	fetch()
navigator	DOM	EVENT
setTimeout		

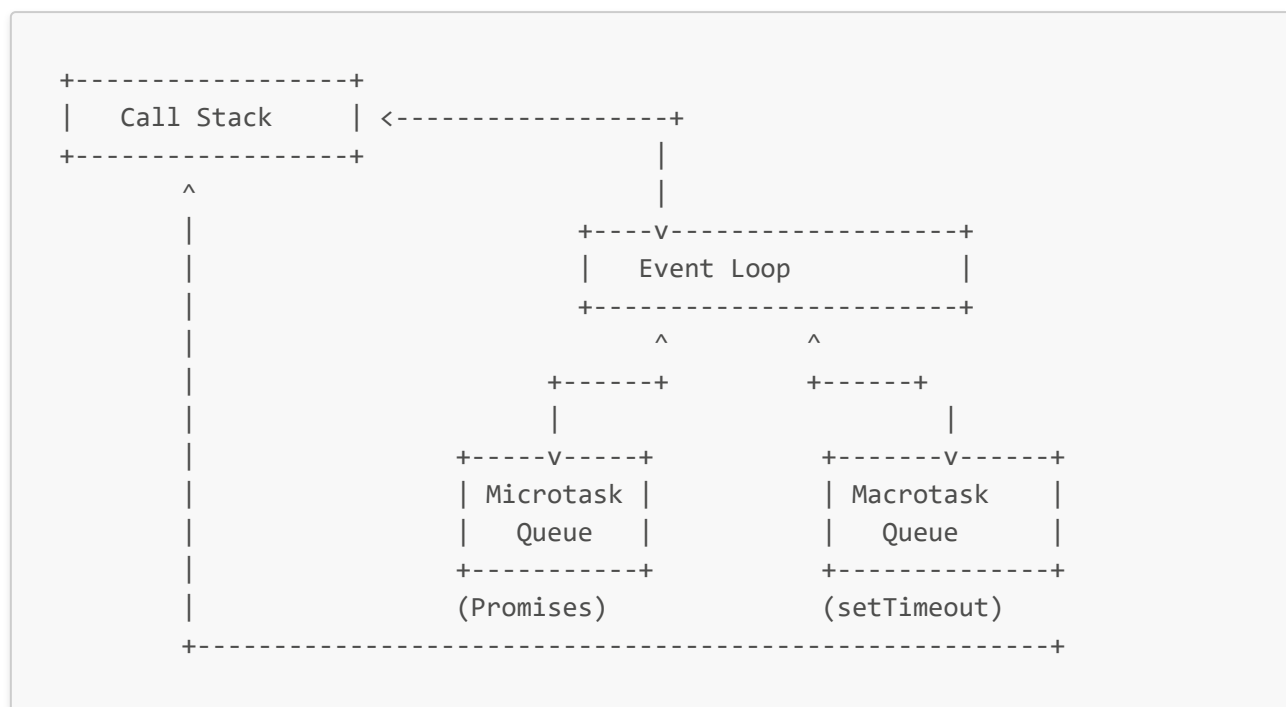
3. Storage APIs

- Local Storage
- Session Storage
- IndexedDB
- Cookies
- Cache
- WebSQL

4. JavaScript Engine Components

- Parser – Checks syntax.
- Abstract Syntax Tree (AST) – Represents structure of code.
- Tokenization – Breaks code into tokens.
- Interpreter – Executes code line-by-line.
- JIT Compiler – Converts code to machine language.
- Garbage Collector – Frees memory.
- Call Stack – Executes and manages function calls (LIFO).
- Heap Memory – Allocates memory for objects.

5. Event Loop



6. Execution Flow

```
console.log("start");
let a = "Ravi";
setTimeout(() => console.log("Timeout"), 2000); // Async (Macrotask Queue)
Promise.resolve().then(() => console.log("Promise")); // Async (Microtask Queue)
console.log(a);
console.log("end");
```

Execution Order:

1. Synchronous code runs first (`start` , variable declarations, `a` , `end`).
2. Microtasks (e.g., Promise).
3. Macrotasks (e.g., `setTimeout` after 2s).

- If a browser has a JS Engine, it becomes a **JRE[Javascript runtime enviroment]**.

JavaScript Execution Model (Synchronous vs Asynchronous)

☑ Synchronous Code:

- All regular JavaScript code (like variable declarations, loops, functions, console logs, etc.) is executed **line-by-line**, one after the other.
- This code goes into the **Call Stack**, which is managed by the **JavaScript Engine** (e.g., V8 in Chrome, SpiderMonkey in Firefox).
- Synchronous code is **blocking**, meaning each statement waits for the previous one to finish.

Example:

```
console.log("Start");  
console.log("End");
```

Output:

```
Start  
End
```

Asynchronous Code (setTimeout, Promises, Events):

◇ `setTimeout()` and similar Web APIs:

- When you use `setTimeout()`, the timer is handled **outside the JS engine** by the **Browser's Web API environment**.
- After the timer completes, the callback function is pushed into the **MACROTASK queue**.

- It waits there until the **Call Stack is empty**, and then the **Event Loop** pushes it to be executed.

◇ Example:

```
console.log("Start");
setTimeout(() => {
  console.log("Inside setTimeout");
}, 1000);
console.log("End");
```

Output:

```
Start
End
Inside setTimeout
```

☑ Here:

- "Start" and "End" run first (synchronously).
- "Inside setTimeout" runs after 1 second, once the call stack is clear.

📚 TASK QUEUES:

1. Microtask Queue

- Contains: `Promise` callbacks, `queueMicrotask()`, `MutationObserver`
- Higher priority than macrotask.

2. Macrotask Queue

- Contains: `setTimeout`, `setInterval`, `setImmediate`, `MessageChannel`, `UI rendering`, etc.

📌 The **Event Loop** constantly checks if the **Call Stack is empty**, and then pushes tasks from the microtask queue first, followed by the macrotask queue.

📄 Summary:

Code Type	Where it goes	Type	Priority
Regular JS Code	Call Stack	Synchronous	High
<code>setTimeout()</code> Callback	Macrotask Queue via Web API	Asynchronous	Low
<code>Promise.then()</code>	Microtask Queue	Asynchronous	Higher

Rendering in JavaScript Engines

Overview

Rendering in JavaScript engines refers to the process of converting JavaScript code and web content into visual output that users can see and interact with in their browsers. However, JavaScript engines themselves don't actually handle rendering - they work closely with rendering engines.

JavaScript Engine vs Rendering Engine

JavaScript Engines

JavaScript engines (V8, SpiderMonkey, JavaScriptCore) are responsible for:

- Parsing and executing JavaScript code

- Memory management
- Garbage collection
- Just-in-time compilation

Rendering Engines

Rendering engines (Blink, Gecko, WebKit) handle:

- Parsing HTML and CSS
- Building the DOM (Document Object Model) and CSSOM (CSS Object Model)
- Layout calculations
- Painting pixels to the screen

How They Work Together

The rendering process involves several key steps where JavaScript and rendering engines collaborate:

1. **DOM Manipulation**

- JavaScript can modify the DOM structure
- Add/remove elements
- Change attributes

2. **Style Changes**

- JavaScript can modify CSS properties
- Change classes
- Modify inline styles

3. Layout Recalculation

- When JavaScript changes affect element dimensions or positions
- The rendering engine recalculates layout

4. Repainting

- Visual changes trigger the rendering engine to repaint affected areas

5. Compositing

- Modern browsers use GPU acceleration to composite layers efficiently

Performance Considerations

Understanding this relationship is crucial for performance because:

- **Expensive Operations:** Frequent DOM manipulations can trigger expensive layout recalculations
- **CSS Properties:** Certain CSS properties are cheaper to animate than others
- **Event Loop:** The browser's event loop coordinates between JavaScript execution and rendering
- **Synchronization:** Techniques like `requestAnimationFrame` help synchronize JavaScript with the browser's refresh rate

Key Insight

While JavaScript engines execute the logic, they rely on separate rendering engines to actually display the results on screen. The collaboration between these two components is what makes modern web applications possible.

Browser Examples

Browser	JavaScript Engine	Rendering Engine
Chrome	V8	Blink
Firefox	SpiderMonkey	Gecko
Safari	JavaScriptCore	WebKit
Edge	Previously use ' chakra ' Now use ' V8 '	Blink
Mozilla Firefox	Spider monkey	Gecko

Node.js Evolution Year-Wise

2009

- **Ryan Dahl** (Creator of Node.js).
- Took **Spider Monkey** (Mozilla's engine) out of the browser.
- Tried for 2 days, failed. Named it **Web.js**.
- Later used **V8 engine** (from Chrome).
- Created **Node.js** – JavaScript runtime for backend.
- Joined company **Joyent**.
- Initially worked only on **Mac and Linux**.

2010

- **Isaac** (Isaac Schlueter) created **npm** (Node Package Manager).
- Joined **Joyent**.

2011

- **Joyent** collaborated with **Microsoft**.
- Released **Node.js for Windows** support.

2012

- **Ryan Dahl** left Node.js project.
- Project handed over to **Isaac**.

2014

- **Fedor**, co-developer of Node.js, left Joyent.
- Created a **forked version of Node.js** named **IO.js**.

2015

- To resolve confusion between Node.js and IO.js, a foundation was created:
 - **Node.js Foundation** established.
 - Merged **Node.js + IO.js** under this foundation.
- Started regular releases.
- Introduced **LTS (Long Term Support)** in 2016:
 - Valid for **30 months / 2.5 years**.
- JS Frontend ecosystem evolved from 2015:
 - ES6
 - Frameworks: **React, Angular, Vue**
 - Feature: **ES Modules** (`import` , `export`)

- Problem: Node.js only supported older CommonJS (`require` , `module.exports`).

2018

- **Node.js v10** released.
- Officially started supporting **ECMAScript Modules (ESM)**.

2019

- **JS Foundation + Node.js Foundation** merged.
- New body formed: **OpenJS Foundation**.

2020

- **Ryan Dahl** returned.
- Introduced **Deno.js** as a **secure competitor to Node.js**.

Feature	Node.js	Deno.js
Uses npm	Yes	No
Package Mgr	npm	Built-in (No installation required)

Modern Module System (Side Note)

ES6 Modules (Frontend & Modern JS)

```
import something from 'module';  
export const data = ...;
```

CommonJS (Old Node.js)

```
const something = require('module');  
module.exports = { ... };
```

Node.js v10+

- Supports both ES6 Modules and CommonJS.
 - Extension `.mjs` for ESM or use `"type": "module"` in `package.json`.
-

Summary Timeline

```
2009 -> Ryan created Node.js using V8 (Joyent)  
2010 -> Isaac created npm  
2011 -> Node.js Windows support via Microsoft  
2012 -> Ryan left, Isaac takes over  
2014 -> Fedor forked to IO.js  
2015 -> Node.js Foundation merges both  
2016 -> LTS support introduced  
2018 -> ES Module support (v10)  
2019 -> OpenJS Foundation created  
2020 -> Ryan returns, introduces Deno
```
